

Introduction to Algorithm Design

HW4

Hasan MEN
131044009

1.

def calcExpo(expo, x) # x^3, x^5, x^4 gibi hesaplama yapıyor.

```
if expo == 1:
    return x
elif expo == 0:
    return 1
```

```
return calcExpo(math.floor(expo/2), x) * calcExpo(math.ceil(expo/2), x)
```

def calcPoly(coefs, expos, x, n): # $2x^3 + 3x^1 + 4 \rightarrow$ (2, 3, 4), (3, 1, 0)

```
res = 0
```

```
for i in range(0, n):
```

```
    res += calcExpo(expos[i], x) * coefs[i]
```

```
return res
```

```
coefArr = [2, 3, 4]
```

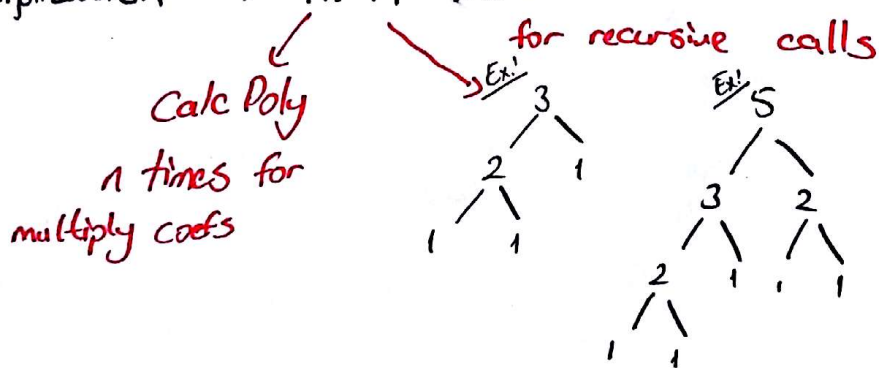
```
expoArr = [2, 3, 0]
```

```
res = calcPoly(coefArr, expoArr, 2, 3)
```

★ Time Complexity = $O(n \log n)$

of summation $\rightarrow n$

of multiplication $\rightarrow n \cdot n = n^2$



```

2. def findEqualIndex (arr, start, end):
    mid = (start + end) / 2
    if start > end:
        return -1
    if mid == arr[mid]:
        return mid
    elif mid > arr[mid]:
        return findEqualIndex (arr, mid+1, end)
    else return findEqualIndex (arr, start, mid-1)

```

Time complexity $O(\log n)$

$$T(n) = 2T(n-1) + O(1)$$

4. Algorithm $\text{MinMax}(\text{arr}[l, r], \text{minval}, \text{maxval})$

if $r = l$
 $\text{minval} \leftarrow \text{arr}[l]$
 $\text{maxval} \leftarrow \text{arr}[l]$

else if $r - l = 1$
 if $\text{arr}[l] \leq \text{arr}[r]$
 $\text{minval} \leftarrow \text{arr}[l]$
 $\text{maxval} \leftarrow \text{arr}[r]$

 else $\text{minval} \leftarrow \text{arr}[r]$
 $\text{maxval} \leftarrow \text{arr}[l]$

else

$\text{MinMax}(\text{arr}[l \dots \lfloor (l+r)/2 \rfloor], \text{minval}, \text{maxval})$

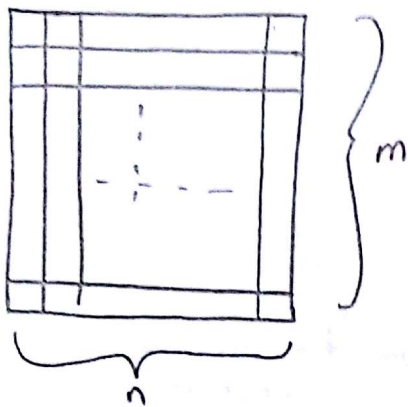
$\text{MinMax}(\text{arr}[\lfloor (l+r)/2 \rfloor + 1 \dots r], \text{minval2}, \text{maxval2})$

 if $\text{minval2} < \text{minval}$
 $\text{minval} \leftarrow \text{minval2}$

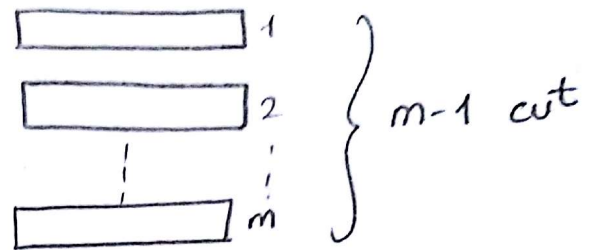
 if $\text{maxval2} > \text{maxval}$
 $\text{maxval} \leftarrow \text{maxval2}$

Copied from text book solutions (sol4-Q2)

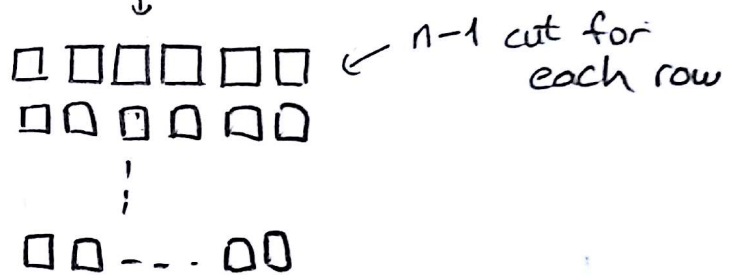
5.



Cut
→



cut
↓



$(m-1)(n-1) \rightarrow$ total cut

→ Cut from rows

→ Cut column for each rows

6.

a) What does dynamic programming have in common with divide-and-conquer?

- Divide a problem into subproblems.
- solve the subproblems
- combine results
- solve bigger problem

b) What is the principal difference between the two techniques?

→ divide-and-conquer → all problems are solved independently
so, we can say all problems are independent. If there are
similar steps, each is solved separately.

→ dynamic programming → solve each subproblems only once
and store their solutions. Then, combines this little solutions
to find big picture. Disadvantage of dynamic programming is
extra space needed. If problem size is n , we need up to n space.

7.

a) $p \rightarrow$ kazanma olasılığı

$q \rightarrow$ kayıma olasılığı

$$\underline{\underline{q = 1 - p}}$$

$$P(i, j) = p \cdot P(i-1, j) + q \cdot P(i, j-1) \quad \text{recurrence relation}$$

$i, j > 0$

A oyun

kazarsa

B oyun

kazarsa

$$P(0, j) = 1$$

$$P(i, 0) = 1$$

b) $p = 0.4$

$$P(1, 1) = \underbrace{0.4}_{1} P(0, 1) + \underbrace{0.6}_{0} P(1, 0) = 0.4$$

$$P(1, 2) = \underbrace{0.4}_{0.4} \underbrace{P(0, 2)}_1 + \underbrace{0.6}_{0.6} \underbrace{P(1, 1)}_{0.4} = 0.24 + 0.24 = 0.48$$

$$P(1, 3) = 0.4 \underbrace{P(0, 3)}_1 + 0.6 \underbrace{P(1, 2)}_{0.48} = 0.288$$

$$P(1, 4) = 0.4 P(0, 4) + 0.6 \underbrace{P(1, 3)}_{0.288} = 0.1728$$

$$P(2, 1) = 0.4 P(1, 1) + 0.6 P(2, 0) = 0.16$$

$$P(2, 2) = 0.4 P(1, 2) + 0.6 P(2, 1) = 0.352$$

$$P(2, 3) = 0.4 P(1, 3) + 0.6 P(2, 2) = 0.528$$

$$P(2, 4) = 0.4 P(1, 4) + 0.6 P(2, 3) = 0.6624$$

$$P(3, 1) = 0.4 P(2, 1) + 0.6 P(3, 0) = 0.064$$

$$P(3, 2) = 0.4 P(2, 2) + 0.6 P(3, 1) = 0.1856$$

$$P(3, 3) = 0.4 P(2, 3) + 0.6 P(3, 2) = 0.3224$$

$$P(3, 4) = 0.4 P(2, 4) + 0.6 P(3, 3) = 0.4632$$

$$P(4, 1) = 0.4 P(3, 1) + 0.6 P(4, 0) = 0.0256$$

$$P(4, 2) = 0.4 P(3, 2) + 0.6 P(4, 1) = 0.09024$$

$$P(4, 3) = 0.4 P(3, 3) + 0.6 P(4, 2) = 0.14912$$

$$P(4, 4) = 0.4 P(3, 4) + 0.6 P(4, 3) = 0.29184$$

i \ j	0	1	2	3	4
0	1	1	1	1	1
1	0	0.4	0.48	0.488	0.4872
2	0	0.16	0.352	0.528	0.6624
3	0	0.064	0.1856	0.3224	0.4632
4	0	0.0256	0.09024	0.14912	0.29184

7. c. Algorithm World Series (n, p)

$q \leftarrow 1 - p$

for $j \leftarrow 1$ to n do

$P[0, j] \leftarrow 1, 0$

for $i \leftarrow 1$ to n do

$P[i, 0] \leftarrow 0, 0$

for $j \leftarrow 1$ to n do

$P[i, j] \leftarrow p * P(i-1, j) + q * P(i, j-1)$

return $P[n, n]$

Efficiency $\Rightarrow \Theta(n^2)$

8- Max Square matrix / Dynamic Programming

0	1	1	0	1
1	1	0	1	0
0	1	1	1	0
1	1	1	1	0
1	1	1	1	1
0	0	0	0	0

ilk sütun ve satırı yedekle
→

0	1	1	0	1
1	x			
0				
1				
1				
0				

⇒ Yedek Dynamic Matrix

X'i bulmak için sol, sağ ve sağ üst komşusuna bak.
Hepsi 1 ise x'te 1 olur ve bir matrix olabilir.
Eğer 1 tane bile 0 varsa x 0 olur.

En sonda oluşan arrayin max indexini bul.

Yedek matrix

0	1	1	0	1
1	1	0	1	0
0	1	1	1	0
1	1	2	2	0
1	2	3	1	
0	0	0	0	0

i=4 j=3 koordinatında 3 birim yukarıda ve aşağıda matrix var.

Asıl Matrix

0	1	1	0	1
1	1	0	1	0
0	1	1	1	0
1	1	1	1	0
1	1	1	1	1
0	0	0	0	0

i=4 j=3

Max Square matrix

```

hasan@hmenn:~/Desktop$ python q8.p
Array to find max square matrix:
01101
11010
01110
11110
11111
00000

->Find maxSquareMatrix
Initial dynamic programming array
01101
10000
00000
10000
10000
00000

Step I: 1 J: 1 subsets
01101
11000
00000
10000
10000
00000

Step I: 1 J: 2 subsets
01101
11000
00000
10000
10000
00000

Step I: 1 J: 3 subsets
01101
11010
00000
10000
10000
00000

Step I: 1 J: 4 subsets
01101
11010
00000
10000
10000
00000

Step I: 2 J: 1 subsets
01101
11010
01000
10000
10000
00000

Step I: 2 J: 2 subsets
01101
11010
01100
10000
10000
00000

Step I: 2 J: 3 subsets
01101
11010
01110
10000
10000
00000

Step I: 3 J: 1 subsets
01101
11010
01110
11000
10000
00000

Step I: 3 J: 2 subsets
01101
11010
01110
11200
10000
00000

Step I: 3 J: 3 subsets
01101
11010
01110
11220
10000
00000

Step I: 3 J: 4 subsets
01101
11010
01110
11220
10000
00000

Step I: 4 J: 1 subsets
01101
11010
01110
11220
12000
00000

Step I: 4 J: 2 subsets
01101
11010
01110
11220
12200
00000

Step I: 4 J: 3 subsets
01101
11010
01110
11220
12230
00000

Step I: 4 J: 4 subsets
01101
11010
01110
11220
12231
00000

Step I: 5 J: 1 subsets
01101
11010
01110
11220
12231
00000

Step I: 5 J: 2 subsets
01101
11010
01110
11220
12231
00000

Step I: 5 J: 3 subsets
01101
11010
01110
11220
12231
00000

Step I: 5 J: 4 subsets
01101
11010
01110
11220
12231
00000

MaxSquareMatrix: Length: 3 ma
xIndexI: 4 maxIndexJ: 3
111
111
111

hasan@hmenn:~/Desktop$

```

```
#!/usr/bin/python
```

```
def printArrayToConsole(arr, startX, startY, endX, endY):
    #print(startX, startY, endX, endY)
    for i in range(startX, startX+endX):
        for j in range(startY, startY+endY):
            print(arr[i][j], end="")
        print("")
    print()

def maxSquareSubset(arr, n, m):
    print("->Find maxSquareMatrix")
    subsets=[[0 for j in range(m)] for i in range(n)] # create an array filled with 0

    for i in range (0,n): # initialize first column
        subsets[i][0] = arr[i][0]

    for j in range (0,m): # initialize first row
        subsets[0][j] = arr[0][j]

    print("Initial dynamic programming array")
    printArrayToConsole(subsets,0,0,n,m)

    for i in range (1,n):
        for j in range(1,m):
            if arr[i][j]==1:
                # check left-up-and left-up cross off item
                # if all of them are 1, we can say there are a matrix
                subsets[i][j]=min(subsets[i][j-1],subsets[i-1][j],subsets[i-1][j-1])+1
            else:
                subsets[i][j]=0
            print("Step I:",i,"J:",j,"subsets")
            printArrayToConsole(subsets,0,0,n,m)

    maxS,maxIndexI,maxIndexJ = subsets[0][0],0,0
    # find max value and index
    for i in range (1,n):
        for j in range(1,m):
            if maxS < subsets[i][j]:
                maxS=subsets[i][j]
                maxIndexI=i
                maxIndexJ=j

    print("MaxSquareMatrix:", "Length:", maxS, "maxIndexI:", maxIndexI, "maxIndexJ:", maxIndexJ)
    printArrayToConsole(arr, maxIndexI-maxS+1, maxIndexJ-maxS+1, maxS, maxS)

myArr = [[0,1,1,0,1],
          [1,1,0,1,0],
          [0,1,1,1,0],
          [1,1,1,1,0],
          [1,1,1,1,1],
          [0,0,0,0,0]]

print("Array to find max square matrix:")
printArrayToConsole(myArr,0,0,6,5)

maxSquareSubset(myArr,6,5)
```

9. Matrix Chain Order - Dynamic Programming

$$m(i, j) = \begin{cases} 0 & \text{if } i=j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_i p_k p_j\} & \text{if } i < j \end{cases}$$

$$A = 10 \times 30 \quad B = 30 \times 5 \quad C = 5 \times 60$$

$$A(BC) = (10 \times 30 + 60) + (30 \times 5 \times 60) = 27000 \quad (AB)C = (10 \times 30 \times 5) + (10 \times 5 \times 60) = 4500$$

$P = [10, 30, 5, 60]$

	1	2	3	4
1	0	0	0	0
2	0	0	1500	4500
3	0	0	0	900
4	0	0	0	0

$\rightarrow \min (m[1, k] + m[k+1, 2] + p_0 p_k p_2)$
 $\rightarrow \min_{2 \leq k < 4} \{m[2, k] + m[k+1, 4] + p_1 p_k p_4\}$
 $k=2/0 + 0 + 10 \times 30 \times 60 = 54000$
 $k=3/1500 + 0 + 10 \times 5 \times 60 = 4500$
 $\dots \dots \dots \underline{\underline{4500 < 54000}}$

$\Rightarrow$ 2. satırın son elemanı optimum değerdir.

$\Rightarrow$ Olası hesaplamalar göz önünde bulundurularak (tek tek) tüm olasılıklara bakılır.

```

matrixchainorder.py x
1 def dynamicMatrixChain(p, n):
2     # array olustur
3     m = [[0 for x in range(n)] for x in range(n)]
4
5     # ilk degerleri 0 olarak ekle
6     for i in range(1, n):
7         m[i][i] = 0
8
9     for length in range(2, n):
10        for i in range(1, n-length+1):
11            j = i+length-1
12            m[i][j] = 99999
13            for k in range(i, j):
14                q = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j]
15                if q < m[i][j]:
16                    m[i][j] = q
17
18        print("\nArray:",p)
19        for i in range(0,n):
20            for j in range(0,n):
21                print(m[i][j], "\t", end="", sep="")
22            print("")
23        return m[1][n-1]
24
25 arr = [10,30,5,60]
26 size = len(arr)
27
28 print("# of Operations ",dynamicMatrixChain(arr, size))
29
30 arr = [40,50,30,10,25]
31 size = len(arr)
32 print("# of Operations ",dynamicMatrixChain(arr, size))
33
34 arr = [6,8,10,20]
35 size = len(arr)
36 print("# of Operations ",dynamicMatrixChain(arr, size))

```

```

hasan@hmenn: ~/Desktop
hasan@hmenn:~/Desktop$ python matrixchainorder.py

Array: [10, 30, 5, 60]
0      |0      |0      |0      |
0      |0      |1500   |4500   |
0      |0      |0      |9000   |
0      |0      |0      |0      |
# of Operations 4500

Array: [40, 50, 30, 10, 25]
0      |0      |0      |0      |0      |
0      |0      |60000  |35000  |45000  |
0      |0      |0      |15000  |27500  |
0      |0      |0      |0      |7500   |
0      |0      |0      |0      |0      |
# of Operations 45000

Array: [6, 8, 10, 20]
0      |0      |0      |0      |
0      |0      |480    |1680   |
0      |0      |0      |1600   |
0      |0      |0      |0      |
# of Operations 1680
hasan@hmenn:~/Desktop$

```

Devrim Arabaları

1960 'lı yıllarda çekilen film dönemin cumhurbaşkanı Cemal Gürsel tarafından istenen Türk yapımı arabanın hikayesini temel almaktadır. O zamanın meşhur sözü "Garp kafasıyla araba yaptık , Şark kafasıyla benzin koymayı unuttuk" bu filmin de konusu olan 23 mühendisin 130 günde iki otomobil yapmalarına dayanmaktadır. Filmde Gündüz isimli mühendisin seçmiş olduğu 22 adet mühendisin 130 günde imkansız olarak bakılan araba yapımı için Eskişehir'deki fabrikada birleşmesi ile başlanmıştır. Devletteki bazı bakanların bu konuda ekonomik sebepleri göz önüne alarak bu işe karşı çıkmalarına rağmen mühendislerin canla başla çalışarak , gazetelerde çıkan karalama haberlere karşın büyük mücadeleyle araba yapımlarını sürdürdükleri gözlenmiştir. Cemal Paşa'nın gelmesini 3 gün önceden haber verilmesi sebebiyle alelacele motoru birleştirmeye çalışmışlardır ve gelmesine 1 gün kala sorun çıktığını gözlemlemişlerdir. Bunun üzerine baş mühendis Gürbüz beyin konuşma yapması ile tekrar canlanan mühendisler Cemal Paşa'nın önüne çalışan bir motor çıkarmayı başarmışlardır. Daha sonrasında büyük zorluklara rağmen araba yapımını tamamlayan mühendisler arabayı trenle götürecekleri ve trende üstüne ateş parçası sıçraması ihtimaliyle benzinini azaltmak zorunda kalmışlardır . Fakat bu durum Cemal Paşa'nın köşkten alınmasıyla devam eden süreçte büyük soruna yol açarak Cemal Paşa'nın köşkten alınıp arabaya bindirildikten 100 metre sonra yolda kalmasına sebep olmuştur. Bunu haber yapan gazetecilerden dolayı halk arabayı sahiplenememiş , arabaların yapımına daha fazla ekonomik destek de bulunmamışlardır. Filmin başında müzeyi ziyaret eden kişi filmin içerisinde rol alan mühendislerden biri olan Cemal'dir. Bu arabaların yapımının durdurulması onu da çok üzmüştür. Arabalardan biri hurdaya çıkmıştır. Sadece bir tanesi Eskişehir'deki müzede durmaktadır.